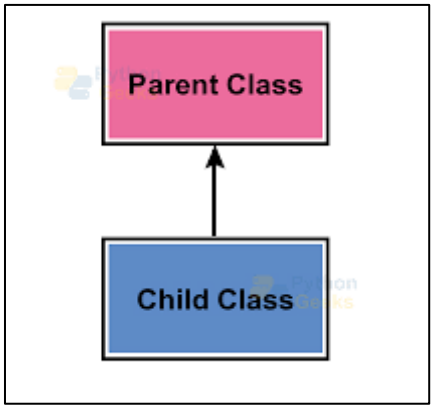
INHERITANCE

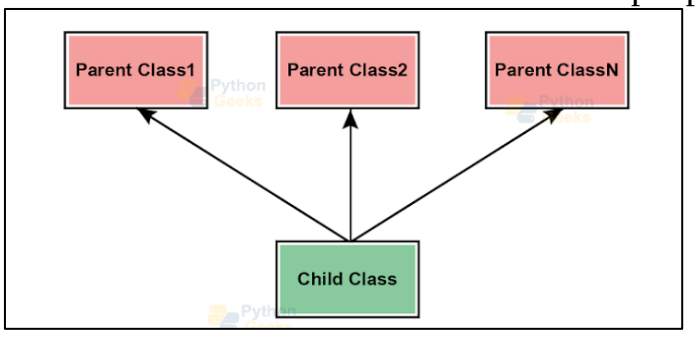
- **Inheritance** allows a new class(child class) to **adopt properties** from an existing class(parent class).
- It promotes **code reusability** by eliminating the need to rewrite duplicate code.

TYPES OF INHERITANCE

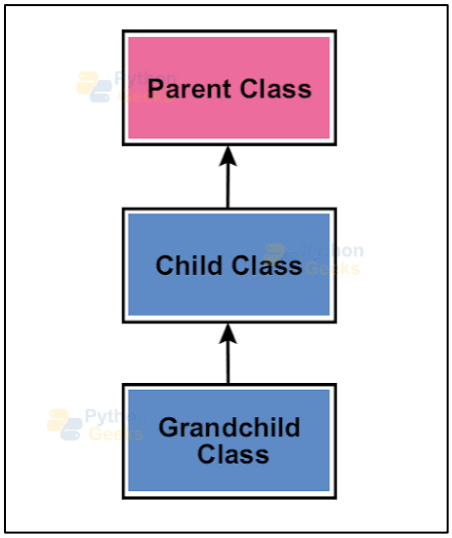
➢ **Single Inheritance**: One child class inherits from one parent class.



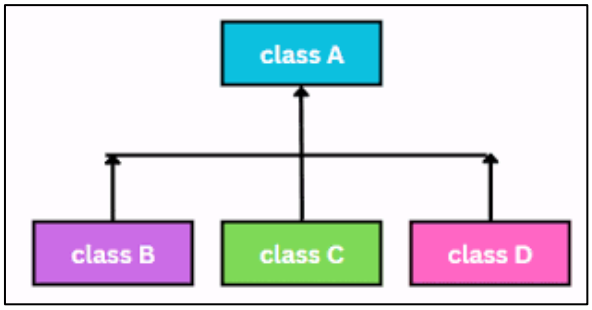
➢ **Multiple Inheritance**: One child class inherits from multiple parent classes.



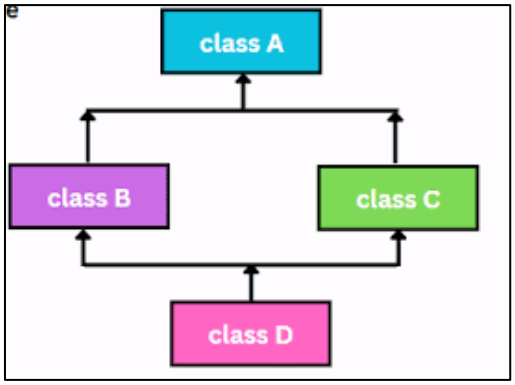
➢ **Multilevel Inheritance**: A child class inherits from a derived parent class, forming a chain.



➢ **Hierarchical Inheritance**: Multiple child classes inherit from a single parent class.



➢ **Hybrid Inheritance**: A combination of two or more types of inheritance.



EXAMPLE

```
interface InterviewSkills {
    void conductInterview();
}

interface BudgetSkills {
    void allocateBudget();
}


class Employee {
    String name;
    public Employee(String name) {
        this.name = name;
    }
    void showName() {
        System.out.println("Employee Name: " + name);
    }
}


class Developer extends Employee {
    String primaryLanguage;
    public Developer(String name, String primaryLanguage) {
        super(name);
        this.primaryLanguage = primaryLanguage;
    }
    void showLanguage() {
        System.out.println(name + " codes in " + primaryLanguage);
    }
}
```

```
class SeniorDeveloper extends Developer {
    int yearsOfExperience;
    public SeniorDeveloper(String name, String primaryLanguage, int
                                                              yearsOfExperience)
{
        super(name, primaryLanguage);
        this.yearsOfExperience = yearsOfExperience;
    }
    void showExperience() {
        System.out.println(name + " has " + yearsOfExperience + " years of
experience.");
    }
}

class Manager extends Employee implements InterviewSkills, BudgetSkills {
    int teamSize;
    public Manager(String name, int teamSize) {
        super(name);
        this.teamSize = teamSize;
    }
    void showTeam() {
        System.out.println(name + " manages a team of " + teamSize + " members.");
    }
    public void conductInterview() {
        System.out.println(name + " is conducting a technical interview.");
    }
    public void allocateBudget() {
        System.out.println(name + " is allocating the project budget.");
    }
}


public class Main {
    public static void main(String[] args) {
        System.out.println("--- 1 & 4. Single & Hierarchical Inheritance Demo ---");
```

```
        Developer dev = new Developer("Alice", "Java");
        dev.showName();
        dev.showLanguage();

        System.out.println("\n--- 2. Multilevel Inheritance Demo ---");
        SeniorDeveloper srDev = new SeniorDeveloper("Bob", "Python", 8);
        srDev.showName();
        srDev.showLanguage();
        srDev.showExperience();

        System.out.println("\n--- 3 & 5. Multiple & Hybrid Inheritance Demo ---");
        Manager mgr = new Manager("Carol", 12);
        mgr.showName();
        mgr.showTeam();
        mgr.conductInterview();
        mgr.allocateBudget();
    }
}
```

FLOW CHART